ICPC Huawei 2026
Edge-Cloud Collaborative Scheduling

An Experimental Optimization Study of an Online Heterogeneous Scheduler

Codeforces 2251A - Engineering Case Study

Nitin Jaiswal
Codeforces: nitinjaiswalnj2001

Final 22-case public score	Earliest exact preserved bank	Final public case #4
16315.7938558663	**16141.8206885046**	**819.0202365952**

Post-contest technical reconstruction
September 2026

Abstract

This report reconstructs an iterative optimization effort for an online edge-cloud scheduling problem implemented in C++17. The preserved scheduler models a serial Edge processor, multiple parallel Cloud processors, FIFO communication links, a one-time Prefill pipeline, and repeated Decode iterations. The optimization objective combines normalized throughput with completion quality derived from Prefill response time (TDR) and token-gap behavior (TPOT). Rather than converging to one universal dispatch rule, the final submission became a portfolio of specialist scheduling behaviors selected by objective regime and workload signature.

The study is organized around controlled public-judge experiments. Major gains came from changing system trajectory rather than only changing local priority: bounded unfinished Prefill work-in-progress produced a 157.956-point gain on public case #15; adaptive Prefill capacity improved case #9; early longest-P_PROC-first admission exposed an early makespan control surface on case #12; and a one-shot early Decode warm phase, later combined with concurrent residual Prefill execution, improved case #4 by 15.268 points. The final publicly verified 22-case score was 16315.7938558663, up 173.973 points from the earliest exact preserved bank of 16141.8206885046.

Negative results are treated as first-class evidence. Several plausible mechanisms produced public no-ops, regressions, or canary failures, showing that local simulation and metric movement were not sufficient grounds for banking a change. The final architecture therefore reflects a disciplined process of identifying live control surfaces, testing the smallest structural intervention, comparing all 22 public cases, and preserving established wins. This report is an engineering case study derived from preserved repository evidence; it is not an official problem statement and does not claim hidden-test optimality.

Keywords: online scheduling, edge-cloud systems, batching, backpressure, work-in-progress control, Decode coherence, experimental optimization, C++17

Evidence and scope note

All numerical claims in this report are drawn from the reconstructed repository's evidence layer. Results labeled public-exact correspond to preserved Codeforces public outputs or exact reconstructed vectors. Public-approx values survive only to limited historical precision. Local-archived results are diagnostic simulator observations and are never treated as banked public improvements. The historical starting score of approximately 15595.199 is retained only as a working-note baseline; the exact 22-case vector for that earliest point was not preserved.

Contents

1. Introduction
2. System model and request lifecycle
3. Objective, metrics, and evidence protocol
4. Baseline architecture and specialist portfolio
5. Study I - Prefill work-in-progress control
6. Study II - Makespan and early control surfaces
7. Negative results and closed branches
8. Study III - Warm Decode phase discovery
9. Study IV - Concurrent warm scheduling
10. Final architecture and public results
11. Robustness and hidden-test considerations
12. Engineering lessons, limitations, and conclusion

Appendix A - Final 22-case public vector

Appendix B - Curated code lineage

Appendix C - Evidence taxonomy and reproducibility

1. Introduction

Online heterogeneous scheduling is difficult when local dispatch choices alter several future queues at once. In the preserved implementation, each request touches a shared Edge resource, communication links, and one assigned Cloud. Prefill must complete before Decode begins, while Decode then repeats for multiple token iterations. Every scheduled batch pays fixed cost, transfers serialize through FIFO links, and work on one resource can create delayed pressure on another. This makes the system sensitive not only to which task runs next, but also to how much work is admitted, how batches are formed, and when phase boundaries are crossed.

The project began with an existing score-aware portfolio rather than a blank scheduler. The engineering objective was therefore incremental but system-level: identify which public cases still had exploitable control surfaces, isolate the bottleneck, change only the mechanism that could plausibly alter that bottleneck, and protect already-strong cases from collateral regressions. This differs from ordinary single-metric tuning. A raw improvement in throughput, TDR, or TPOT could be score-neutral if normalization was saturated or the corresponding objective weight was small.

The most important methodological choice was to treat the public judge as the authoritative instrument. Archived local cases were used to debug mechanisms, map threshold responses, and eliminate obvious failures. But a candidate entered the bank only after public verification and a full 22-case comparison. Over time, exact public no-ops became useful observations: they showed that many apparently reasonable priority controls were not live in the relevant trajectory.

Primary research question
Which scheduler decisions still have enough horizon to change the global resource trajectory, and can those decisions improve the weighted public objective without damaging protected cases?

2. System model and request lifecycle

The reconstructed system contains one serial Edge processor, K parallel Cloud processors, and two serialized communication directions. A request is assigned to a Cloud during Prefill and retains that Cloud for subsequent processing. The same request then follows a one-time Prefill lifecycle followed by repeated Decode iterations.

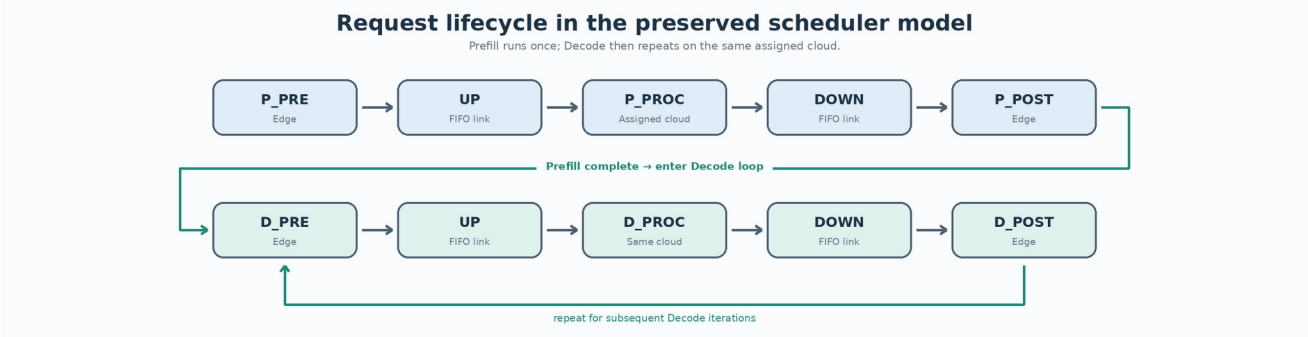


Figure. Request lifecycle in the preserved scheduler model.

Evidence: docs/architecture/SYSTEM_MODEL.md and REQUEST_LIFECYCLE.md

2.1 Prefill pipeline

Prefill consists of P_PRE on the Edge, an UP transfer, P_PROC on the assigned Cloud, a DOWN transfer, and P_POST on the Edge. The implementation records TDR when P_POST completes, so admission and backpressure decisions that accelerate or delay completion of admitted Prefill chains directly affect the completion component of the objective.

2.2 Decode loop

After P_POST, a request becomes ready for D_PRE. Decode then cycles through D_PRE on the Edge, an UP transfer, D_PROC on the same Cloud, a DOWN transfer, and D_POST on the Edge. D_POST increments the produced-token count and updates the previous-token completion timestamp used to compute TPOT gaps. The first produced token initializes the timestamp rather than adding a previous-token gap.

2.3 Why batching and coherence matter

Each scheduled task pays a fixed scheduling cost. Decode batches can therefore amortize overhead, but excessive fragmentation can destroy efficiency. At the same time, waiting for a perfectly coherent generation can leave resources idle and delay useful work. The later stages of the project can be understood as a search for the right compromise between coherence, early start, and opportunistic overlap.

2.4 Compact state machine

Compact state	Meaning
Z15	P_PRE ready
Z36	P_PRE running
Z21	Prefill UP pending
Z8	P_PROC ready
Z22	P_PROC running
Z51	Prefill DOWN pending
Z27	P_POST ready
Z59	P_POST running
Z9	D_PRE ready
Z52	D_PRE running
Z20	Decode UP pending
Z16	D_PROC ready
Z75	D_PROC running
Z53	Decode DOWN pending
Z23	D_POST ready
Z60	D_POST running
Z39	finished

3. Objective, metrics, and evidence protocol

3.1 Weighted normalized objective

The final scheduler reasons about normalized throughput quality and normalized completion quality. The preserved implementation combines them approximately as:

```
utility ≈ wTp × norm_tp + wC × norm_c
```

Throughput normalization uses an instance-specific baseline and upper reference. Completion normalization measures how far mean TDR and mean TPOT exceed their respective SLO references, combines those excesses through Euclidean distance, and normalizes that distance against an instance reference. Consequently, the same raw metric delta can have very different score impact across cases.

```
norm_tp = clamp((tp - tpBase) / (tpUB - tpBase), 0, 1)

excess_tdr  = max(0, mean_tdr  / SLO1 - 1)
excess_tpot = max(0, mean_tpot / SLO2 - 1)
dist        = hypot(excess_tdr, excess_tpot)
norm_c      = max(0, 1 - dist / distBase)
```

3.2 Experimental control loop

Optimization proceeded through repeated system identification rather than broad heuristic search. Public outputs were first interpreted to identify a likely bottleneck. A controlled variant was then built around one hypothesized live control surface. Local cases were used only to diagnose mechanism and obvious failure. The public judge then determined whether the surface was genuinely active.

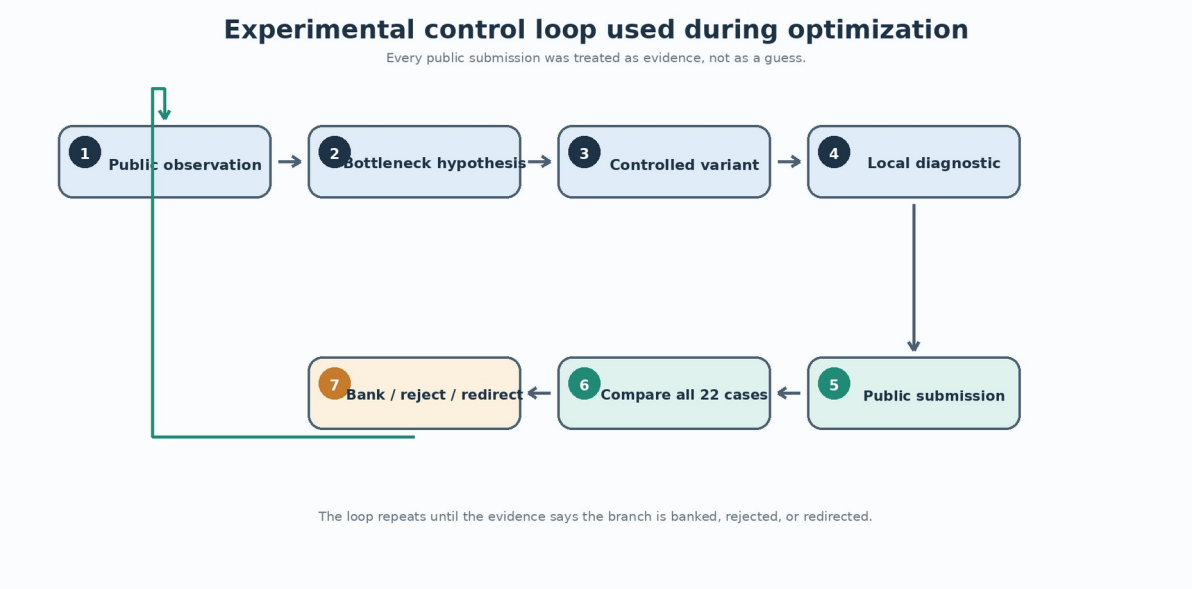


Figure. Experimental control loop used during optimization.

Evidence: docs/methodology/EXPERIMENTAL_METHOD.md

3.3 Evidence taxonomy

Evidence class	Interpretation
public_exact	Exact public result or exact reconstructed public vector is preserved.
public_approx	Public behavior is known, but an older value survives only to limited precision.
local_archived	Archived local simulator result; diagnostic only, never banked by itself.
historical_note	Working observation retained without a complete exact artifact.

3.4 Protected-case discipline

As the portfolio improved, the cost of collateral regression increased. Public case #20, fixed at 998.0785092863 in the banked vectors, became an effective canary. Case #13 was also treated cautiously because historical experiments showed sensitivity to code/layout changes. A target-case improvement was therefore insufficient on its own; all 22 cases had to be inspected before a branch was banked.

4. Baseline architecture and specialist portfolio

The final submission is not one monolithic priority function. It is a portfolio that routes different objective regimes and workload signatures to retained specialist behaviors. The preserved architecture contains a NOVA-style generic scheduler, B1 completion/makespan logic, C2 coherent Decode-wave behavior, Prefill backpressure specialists, a Warm13 path, and PhaseScheduler logic used by the final case #4 regime.

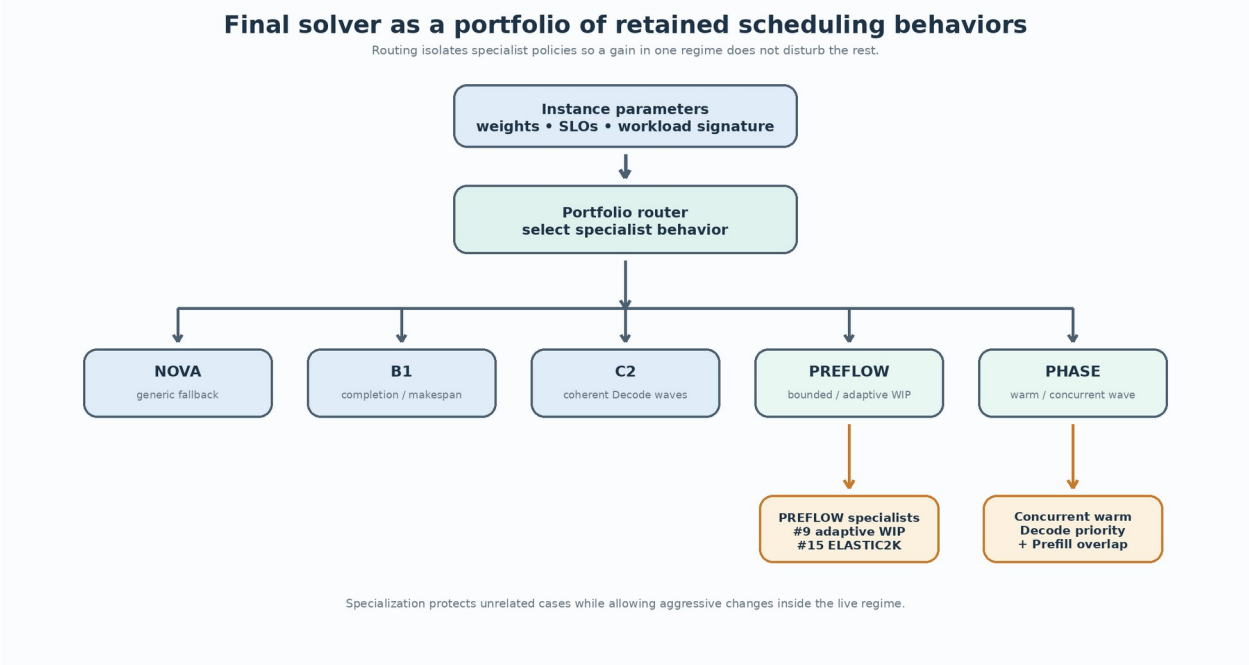


Figure. High-level final scheduler portfolio.

Evidence: docs/architecture/SCHEDULER_ARCHITECTURE.md and final/final_submission.cpp

This specialization was an empirical consequence of public heterogeneity. Some cases rewarded completion and flow control; others behaved like pure makespan; some required large coherent Decode generations; and #4 benefited from an early but carefully overlapped Decode phase. Attempts to force a universal policy repeatedly either produced no-op behavior or damaged already-strong cases.

4.1 Earliest exact preserved bank

The earliest exact full 22-case bank retained for the architecture work scored 16141.8206885046. Earlier references exist, including an approximate historical starting score near 15595.199 and preserved NOVA/SPRINTK2 source checkpoints, but the original exact 22-case vector for the earliest baseline was not recovered. This report therefore uses 16141.8206885046 as the exact quantitative starting point for bank-to-final gain calculations.

5. Study I - Prefill work-in-progress control

5.1 Case #15: bounded unfinished Prefill chains

Case #15 produced the largest single public improvement in the project. The initial behavior scored 714.7974317905. The key hypothesis was that the scheduler was admitting too many unfinished Prefill chains, causing whole-system queue growth even when individual dispatch choices looked reasonable. The intervention was therefore not a new priority comparator; it was a cap on unfinished Prefill work-in-progress from P_PRE admission through P_POST completion.

Variant	#15 public score	Decision
Unbounded	714.797432	baseline
W4K	869.074583	candidate
W3K	869.561999	candidate
W2K	872.705816	banked
ELASTIC2K	872.753788	banked / best
Tail widen	868.885495	rejected

The response curve made the mechanism clear. Bounding WIP produced a jump of more than 154 points even at W4K, while W2K reached 872.705816. A small elastic starvation-relief rule improved that to 872.7537879351. The final gain

on #15 was 157.9563561446 points. The important lesson was that queue population itself was a control variable: the scheduler could not repair an overloaded trajectory merely by choosing a better next task inside the overload.

5.2 Case #9: phase-dependent capacity

The same backpressure concept generalized to #9 but required a different capacity profile. Static widths showed a useful region near W2K, while W1K collapsed from 735.991312 to 591.359356 through starvation. The best public version therefore started at 2K and widened permanently to 4K after sufficient Prefill progress. This improved #9 to 736.7399416791 without disturbing the protected portfolio.

Study I conclusion
Backpressure was not merely a congestion guard. It changed the future completion trajectory by limiting the number of incomplete Prefill chains competing for Edge, Cloud, and link resources. The best capacity could also depend on phase: tight startup control and wider steady-state capacity were compatible.

6. Study II - Makespan and early control surfaces

Public case #12 behaved almost like a throughput/makespan objective. The strongest live surface was early P_PRE admission order. A longest-P_PROC-first rule, combined with least-loaded cloud placement, moved the case from approximately 799.108298 to 800.4563453039. Later attempts were useful mainly because they showed where the control surface stopped being live.

Experiment	Public outcome	Decision
Early LPT admission	800.456345	banked
SPT after admission 16	800.456345	exact no-op
SPT after admission 8	799.102504	regression
Pure SPT	799.104036	regression
Early beam probe	800.456345 on #12; #20 fell to 997.771060	rejected

The contrast between the admission-16 no-op and admission-8 regression suggested that the critical decision horizon lay early in the request stream. Once the first LPT-shaped prefix had established the resource trajectory, later reorderings often had insufficient horizon to change makespan. This became a general principle used elsewhere: search for decisions that are still causally upstream of the bottleneck rather than optimizing late-stage details that the system has already committed to.

7. Negative results and closed branches

A substantial fraction of the useful information came from rejected branches. Negative results reduced the plausible search space, protected the bank, and exposed which metrics were or were not controlling public score. Four groups were especially informative.

Branch	Target	Observation	Conclusion
WAVEPIPE56 / WAVEBOUND56	#5/#6	Fragmenting D_PRE or advancing D_POST broke generation coherence; #6 lost roughly 12 points in one fragmented probe.	Close branch; preserve coherent Decode waves.
PREFLOW10	#10	Promising local completion predictor became a public no-op / tiny negative.	Do not bank local-only gain.
CADENCE3 / TDRLANE3	#3	Active Decode caps changed throughput/TPOT while TDR remained unchanged; W1K Prefill WIP collapsed #3 to 202.678.	Close simple WIP/cadence interpretation.
EARLYBEAM12	#12 / #20	#12 stayed exact while protected #20 moved from 998.078509 to 997.771060.	Reject broad branch; preserve canary.

These failures also demonstrate why local simulation could not be treated as a substitute for public verification. Local cases were valuable for mechanism debugging, but some local gains disappeared completely on the judge, while apparently tiny structural changes could move sensitive protected cases. The process therefore evolved toward smaller public probes with explicit kill criteria.

8. Study III - Warm Decode phase discovery

The next major breakthrough came from recognizing a phase barrier on case #4. The original PhaseScheduler effectively waited for a very coherent transition from Prefill into Decode. A one-shot warm rule allowed the first Decode cohort to begin before every Prefill request had fully drained, provided enough first-token-ready requests had accumulated and none of them had already produced Decode tokens.

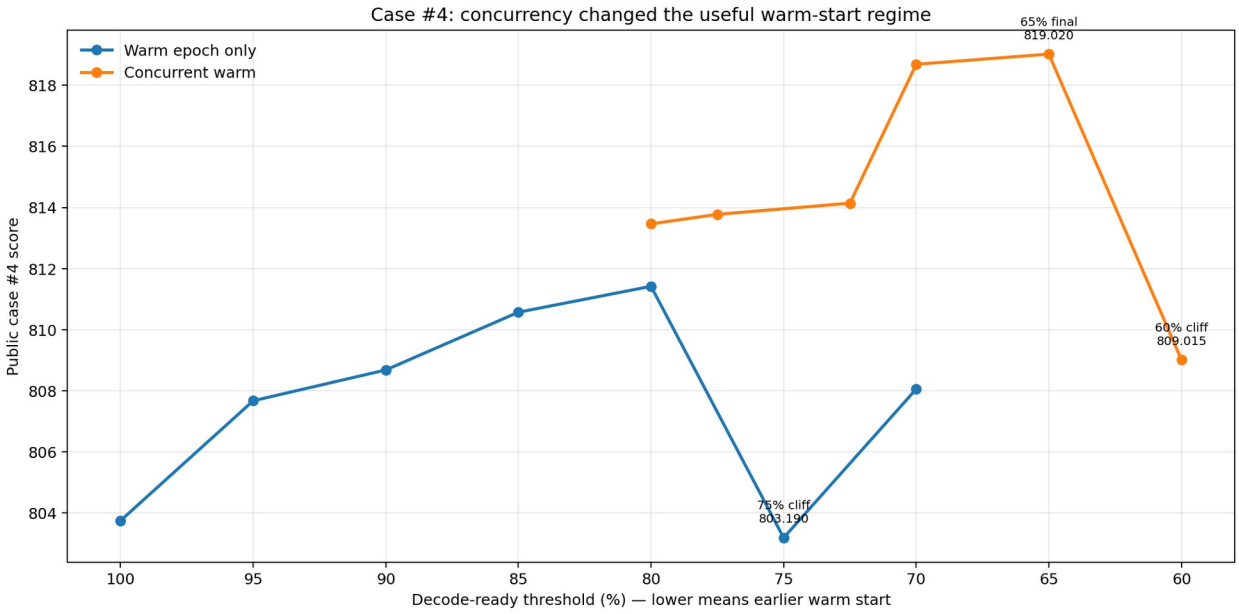


Figure. Public case #4 score across non-concurrent and concurrent warm thresholds.

Evidence: experiments/data/case4_threshold_curve.csv

The non-concurrent threshold curve initially looked monotonic: 95%, 90%, 85%, and 80% progressively improved the score. The 80% variant reached 811.4206018252, a gain of 7.668547 points over the original phase behavior. But 75% collapsed to 803.1898710707, while 70% recovered only partially. This was a discrete schedule transition rather than a smooth coefficient optimum.

Warm readiness	#4	TP	TDR	TPOT	Status
100%	803.752055	0.059507	465.245	84.904	diagnostic
95%	807.673404	0.061011	467.337	85.506	banked
90%	808.683300	0.061498	469.884	85.941	candidate
85%	810.572301	0.062260	471.631	85.353	banked
80%	811.420602	0.062664	473.665	85.580	banked
75%	803.189871	0.060232	483.828	91.683	rejected
70%	808.058931	0.062253	489.502	97.131	rejected

The 75% cliff changed the hypothesis. The problem was no longer simply 'what is the best warm threshold?' Starting Decode earlier was useful, but the warm epoch was reserving resources too aggressively and delaying residual Prefill. The next design question became whether Decode coherence could be preserved without monopolizing Edge and Cloud resources that the cohort was not currently able to use.

9. Study IV - Concurrent warm scheduling

Concurrent warm scheduling changed the semantics of the warm epoch. The Decode cohort retained priority, but idle holes were filled by residual Prefill: the Edge could execute a ready P_POST when the epoch had no D_POST ready, and a Cloud could execute residual P_PROC when the epoch had no D_PROC ready on that Cloud. This retained the batching/coherence benefit of an explicit cohort while removing unnecessary phase serialization.

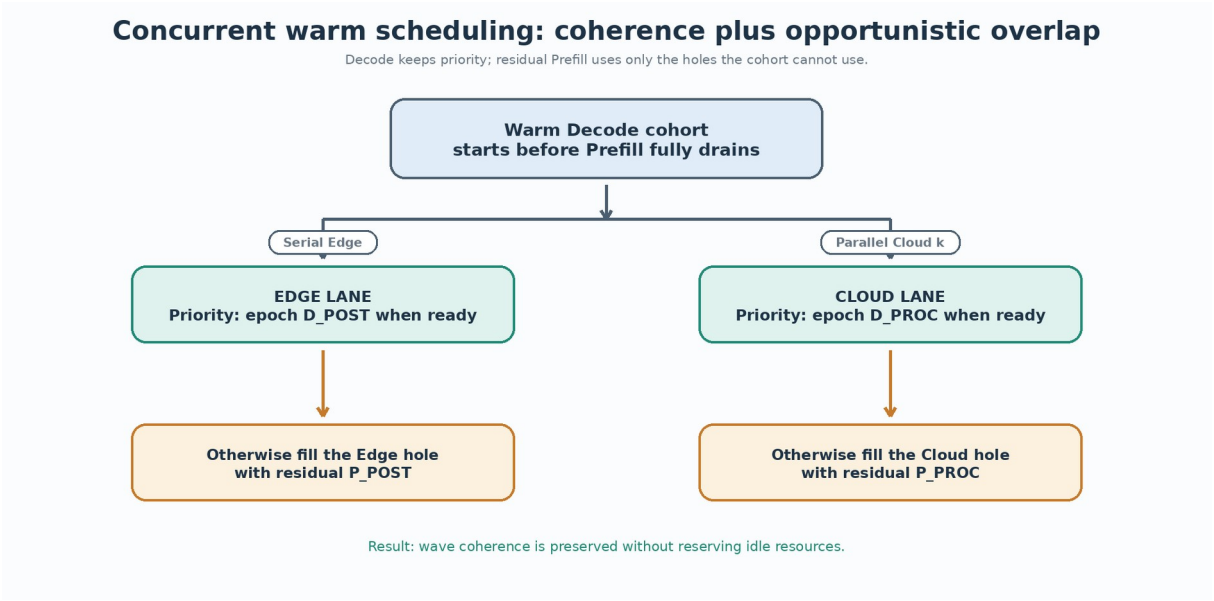


Figure. Concurrent warm pipeline: Decode coherence with residual Prefill overlap.

Evidence: docs/architecture/FINAL_CONCURRENT_WARM_PIPELINE.md

9.1 CONCURRENT80: simultaneous throughput and completion improvement

Variant	#4 score	TP	mean TDR	mean TPOT
Non-concurrent 80%	811.420602	0.062664	473.665006	85.580454
Concurrent 80%	813.462929	0.062973	465.245343	84.083242

This comparison was unusually strong because the concurrent architecture improved both sides of the objective. Throughput rose, TDR returned to 465.245343, and TPOT improved. The mechanism was therefore not merely accepting worse completion in exchange for more throughput; it was reducing idle-resource serialization created by the earlier warm-phase semantics.

9.2 Reopening the early-start region

Once Prefill could continue underneath the warm Decode cohort, thresholds that had been unsafe became useful again. Public case #4 increased gradually through 77.5% and 72.5%, then jumped sharply at 70% to 818.6873903895. A 65% threshold improved again to 819.0202365952, while 60% collapsed to 809.0146923137. The architecture had moved the safe boundary substantially earlier but had not removed threshold cliffs entirely.

Concurrent readiness	#4	TP	TDR	TPOT	Status
80%	813.462929	0.062973	465.245	84.083	banked
77.5%	813.775135	0.063085	465.245	83.895	candidate
72.5%	814.140250	0.063215	465.245	83.567	candidate
70%	818.687390	0.065020	468.893	86.878	banked
65%	819.020237	0.065139	468.893	86.339	final
60%	809.014692	0.061553	468.599	86.355	rejected

Study IV conclusion
The final case #4 mechanism combined three ideas: coherent Decode cohort formation, an early first Decode start, and opportunistic Prefill overlap. Coherence and concurrency were not opposites; the scheduler could preserve cohort priority without reserving resources the cohort could not currently use.

10. Final architecture and public results

The final publicly verified solver retained the specialist portfolio and used the 65% concurrent warm threshold for the final #4 regime. The total public score was 16315.7938558663. Relative to the earliest exact preserved bank of 16141.8206885046, the exact bank-to-final gain was 173.9731673617 points. The approximate historical starting score near 15595.199 suggests a larger overall journey, but that earliest vector is not used for exact arithmetic.

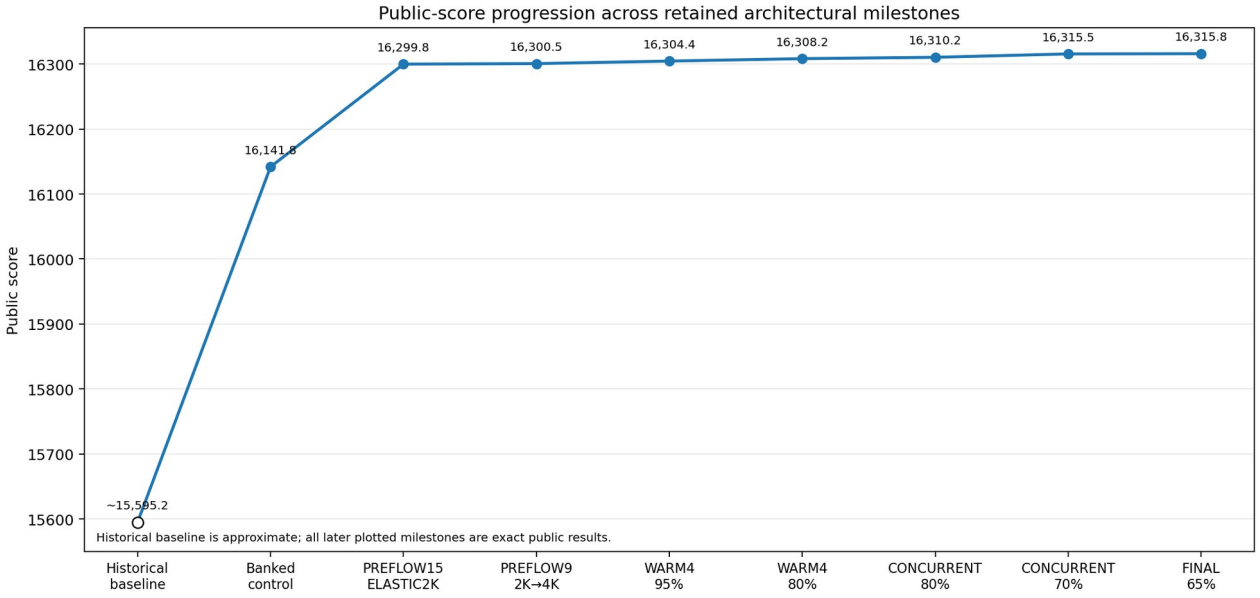


Figure. Public score progression across retained milestones.

Evidence: experiments/data/score_progression.csv

10.1 Major public breakthroughs

Case	Earlier	Final / banked	Gain	Main mechanism
#4	803.752055	819.020237	+15.268182	One-shot warm Decode epoch + concurrent residual Prefill overlap
#9	735.991312	736.739942	+0.748629	Adaptive Prefill WIP: 2K startup → 4K after Prefill progress
#12	799.108298	800.456345	+1.348047	Early longest-P_PROC-first (LPT) admission
#15	714.797432	872.753788	+157.956356	Bounded Prefill WIP / completion conveyor with elastic starvation relief

10.2 Final vector characteristics

The final portfolio remained heterogeneous. Some cases were throughput-dominated, some completion-sensitive, and others nearly saturated on one component. The bank therefore reflects successful coexistence rather than convergence toward a single global heuristic. Case #20 remained at 998.0785092863 and continued to serve as a useful stability indicator. Case #4 reached 819.0202365952 through the concurrent-warm path, while the large #15 backpressure gain remained intact.

10.3 Per-case public score profile

The final vector spans very different score regimes, which reinforces the portfolio interpretation. Cases #16, #20, #21, and #22 are near the upper end of the public scale, while cases such as #5, #6, and #14 retain much lower point values. The optimization work did not attempt to flatten this distribution with one universal rule; instead, it concentrated on public cases where a live and isolatable structural control surface could be identified without sacrificing protected wins.

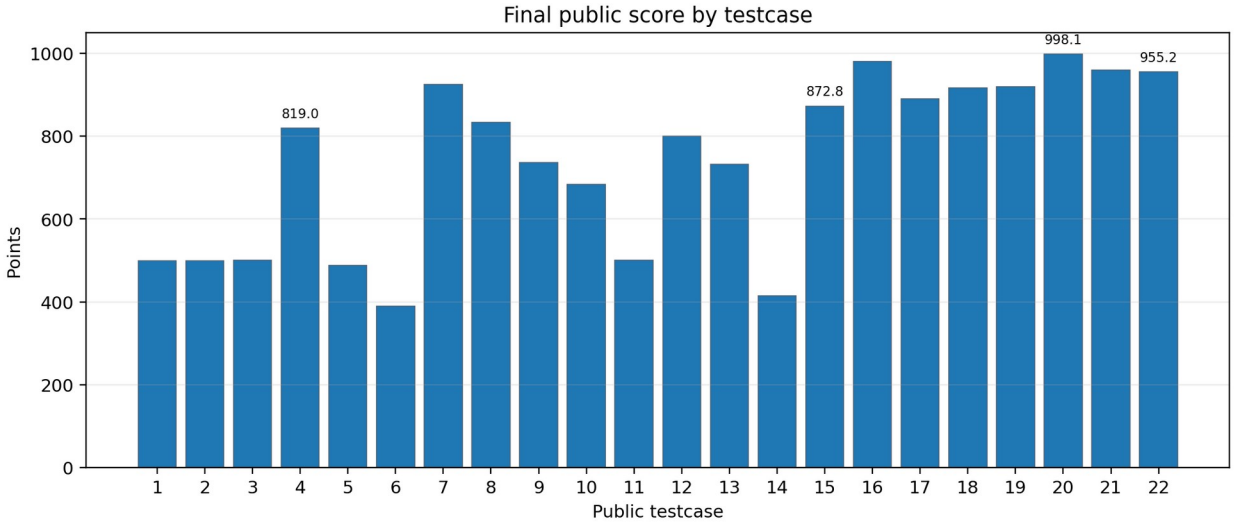


Figure. Final public points by testcase.

Evidence: experiments/data/final_public_22_cases.csv

10.4 Final submission provenance

The exact algorithmic 65% source is preserved separately from the actual Codeforces resubmission. The resubmission differs only by a harmless end-of-file comment added because identical source text could not be submitted again. The repository retains both source hashes and a screenshot of the final public submission for provenance.

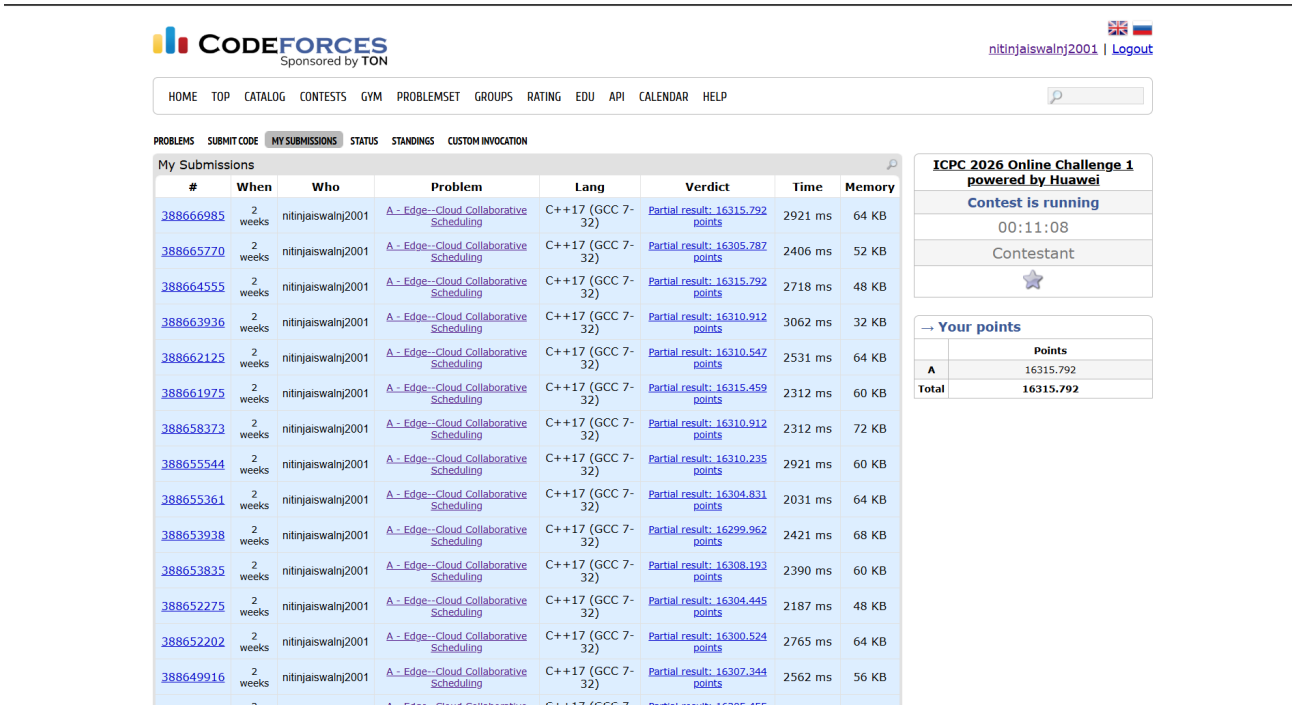


Figure. Preserved screenshot of the final public Codeforces submission and score.

Evidence: provenance/final_submission_codeforces.png

11. Robustness and hidden-test considerations

Public optimization cannot guarantee hidden-test behavior. The late-stage strategy therefore shifted from exploration toward preservation. The final build was not modified with speculative guards after the 65% result. Instead, the exact publicly verified behavior was re-submitted so that the selected positive-score submission corresponded to a known bank rather than an unverified last-minute heuristic.

1. Protect specialist routes that had already produced exact public gains.
2. Treat case #20 as a canary for unintended changes outside the target regime.
3. Avoid broad refactors late in the contest because case #13 had shown sensitivity to code/layout changes in historical work.
4. Prefer the best publicly verified threshold over extrapolating beyond a known cliff.
5. Keep local-archived evidence separate from banked public evidence.

This conservative ending is important to the engineering interpretation. The final scheduler is not claimed to be globally optimal or hidden-test safe in a formal sense. Rather, it is the best publicly verified point in a sequence of controlled structural experiments, with explicit protection of earlier wins.

11.1 Threats to validity

Three validity threats remain. First, public-case specialization can overfit the visible distribution; the portfolio mitigates but cannot eliminate this risk. Second, archived local cases are an imperfect model of the public judge and were observed to produce false-positive signals. Third, the reconstructed history is complete only where artifacts survived: several early branches and the ~15595.199 starting baseline are known from working notes but lack full exact vectors. For these reasons, causal claims in this report are limited to mechanisms supported by controlled public comparisons or by direct implementation semantics.

11.2 Final selection rule

The final decision rule was deliberately asymmetric: a candidate needed evidence of improvement to enter the bank, but once a strong bank existed, an unverified late change was assumed risky until proven otherwise. This asymmetry is appropriate near a contest deadline because the cost of losing a known positive submission can exceed the expected value of a speculative final experiment. The 65% concurrent build was therefore re-submitted unchanged in behavior and then frozen.

12. Engineering lessons, limitations, and conclusion

12.1 Engineering lessons

Control work-in-progress, not only dispatch priority. The #15 result showed that queue population can dominate system behavior. Once too much unfinished work is admitted, local priority changes may be unable to repair the trajectory.

Capacity can be phase dependent. The #9 2K-to-4K design showed that startup and steady-state can need different WIP limits even under the same basic backpressure mechanism.

Early decisions can be more valuable than late decisions. Case #12 showed that the first admission prefix remained live while later reorderings were often exact public no-ops.

No-ops are measurements. A public no-op rules out a control surface just as a gain validates one; this prevented repeated tuning of inactive mechanisms.

Coherence and concurrency can coexist. The final #4 architecture kept a coherent Decode cohort but opportunistically filled unused resource slots with residual Prefill.

Thresholds can switch discrete regimes. The 75% non-concurrent, 70% concurrent, and 60% concurrent transitions showed that a small parameter change can form a qualitatively different epoch.

Public verification outranks local intuition. Archived local cases were useful for diagnosis, but the bank changed only when the public judge confirmed the change.

12.2 Limitations

- The earliest historical baseline (~15595.199) is approximate because its exact 22-case vector was not preserved.
- The report reconstructs the implementation and public experiment history; it is not an official contest specification.
- Public-case specialization creates a risk of overfitting, although protected-case and hidden-test discipline were used to reduce late-stage risk.
- Some historical branches survive only as working-note observations rather than complete source-plus-vector artifacts.
- The final compacted C++ source prioritizes contest source-budget constraints over readability; the surrounding documentation is required to understand the design cleanly.

12.3 Conclusion

The final score was not produced by one isolated trick. The strongest improvements came from repeatedly asking which resource trajectory was actually controlling the current case, which scheduler decision still had enough horizon to alter that trajectory, and whether the public judge confirmed that the proposed surface was live. The largest gain came from Prefill WIP control; the most instructive architectural gain came from the concurrent warm pipeline, which reconciled early Decode coherence with continued Prefill progress.

The resulting portfolio demonstrates a broader systems lesson: online scheduling quality often depends less on the local ordering of already-ready tasks than on controlling admission, phase transitions, batch coherence, and resource overlap. Negative experiments were essential because they exposed inactive controls and protected the search from local overfitting. The repository and this report preserve that evidence so that the final submission can be read as an engineering process rather than as an opaque contest artifact.

Appendix A - Final 22-case public vector

The table below reproduces the final exact public vector preserved in experiments/data/final_public_22_cases.csv. Values are included primarily for reproducibility and future comparison.

Case	Points	TP	mean TDR	mean TPOT	norm_tp	norm_c
1	500.000003	0.022222	30.0	0.0	0.0	1.0
2	500.000000	0.005755	126.158679	0.0	0.0	1.0
3	500.567967	0.004432	1329.849832	61.933452	0.615102	0.500568
4	819.020237	0.065139	468.892661	86.339424	0.528951	0.943336
5	487.783355	1.212616	1497.254452	62.438988	0.360295	0.997736
6	389.860520	0.696973	3102.23197	57.810294	0.32295	0.992051
7	924.498223	0.008881	858.868074	62.657461	0.211544	0.924498
8	833.076958	0.013218	1086.885839	97.967553	0.764356	0.855984
9	736.739942	0.004395	5714.101567	0.0	0.962219	0.724873
10	684.453465	0.007629	182521.129892	88.94038	0.994438	0.62975
11	500.185382	7e-06	32780482.884393	16199.089335	0.000371	1.0
12	800.456345	2.4e-05	1566512.241418	636.942355	0.808542	0.0
13	732.373915	0.027115	1669.941409	71.353839	0.69422	0.846836
14	415.266866	0.003564	192.489397	184.499328	0.210323	0.795876
15	872.753788	9e-06	7899811.503184	0.0	0.965478	0.796889
16	980.266623	0.029775	41823.989666	77.322508	0.981664	0.911808
17	890.234278	0.000521	18892825.775249	23528.736646	0.98901	0.68969
18	916.203378	9e-06	17713632.378469	0.0	0.991936	0.811621
19	919.530340	0.687432	168.439488	181.835358	0.91953	0.999937
20	998.078509	0.005607	1256.98193	208.740759	0.995271	0.999799
21	959.228177	0.012468	51283.856513	0.0	0.992213	0.926244

Case	Points	TP	mean TDR	mean TPOT	norm_tp	norm_c
22	955.215585	39.871593	1858.0	6.003124	0.913514	0.996917

Exact total: 16315.7938558663

Appendix B - Curated code lineage

The public repository contains selected architectural milestones rather than every scratch variant. This keeps the code history readable while the experiment ledger retains rejected and no-op branches.

Stage	File	Role	Public total	Status
00_baseline	nova_portfolio_16024_reference.cpp	earliest preserved NOVA/portfolio reference	-	reference
00_baseline	sprintk2_banked_16042_reference.cpp	score-aware portfolio checkpoint	~16042.0524	historical checkpoint
01_portfolio_scheduler	banked_16141_8207.cpp	exact banked control	16141.8206885046	banked
01_portfolio_scheduler	lexical_compaction_exact.cpp	source-size compaction	16141.8206885046	banked infrastructure
02_preflow15	preflow15_elastic2k.cpp	bounded Prefill WIP	16299.7770446492	banked
03_preflow9	preflow9_adaptive_2k4k.cpp	adaptive Prefill WIP	16300.5256739172	banked
04_makespan12	makespan12_v1_lpt.cpp	early LPT admission	-	banked specialist
05_phase_warm	warm4_95_bank_16304_4470.cpp	one-shot ~95% warm Decode epoch	16304.4470237562	banked
05_phase_warm	warm4_ready85.cpp	85% warm threshold	16307.3459205415	banked
05_phase_warm	warm4_ready80.cpp	80% non-concurrent warm threshold	16308.1942210963	banked
06_concurrent_warm	concurrent80_v1.cpp	concurrent residual Prefill overlap	16310.2365487077	banked
06_concurrent_warm	concurrent70_v2.cpp	70% concurrent warm threshold	16315.4610096606	banked
06_concurrent_warm	concurrent65_final_verified.cpp	65% concurrent warm threshold	16315.7938558663	final verified
06_concurrent_warm	concurrent60_boundary_rejected.cpp	60% boundary probe	-	rejected

Appendix C - Evidence taxonomy and reproducibility

C.1 Repository evidence map

Artifact	Repository path
Final submission	final/final_submission.cpp
Canonical experiment ledger	experiments/data/experiment_ledger.csv
Final public vector	experiments/data/final_public_22_cases.csv
Score progression	experiments/data/score_progression.csv
Case #4 threshold curve	experiments/data/case4_threshold_curve.csv
System model	docs/architecture/SYSTEM_MODEL.md
Scheduler architecture	docs/architecture/SCHEDULER_ARCHITECTURE.md
Concurrent warm design	docs/architecture/FINAL_CONCURRENT_WARM_PIPELINE.md
Experimental method	docs/methodology/EXPERIMENTAL_METHOD.md
Optimization journey	docs/journey/01_JOURNEY_OVERVIEW.md
Final provenance	provenance/

C.2 Validation commands

```
python3 experiments/validate_dataset.py

g++ -std=c++17 -O2 -pipe -fsyntax-only final/final_submission.cpp
```

At Stage 6, the dataset validator reported 111 checks, 111 passes, and 0 failures. The final source also passed C++17 syntax compilation. Stage 7 repeats these gates after report integration.

C.3 Reconstructed-history policy

Git history for the contest itself was not available from the beginning of the optimization effort. The repository therefore presents an openly reconstructed engineering history based on preserved source snapshots, public judge outputs, archived local experiment artifacts, and hashes. It does not backdate commits or imply that the curated repository structure existed during the contest.

End of report